JVD Internal Management System

Comprehensive QA Audit Report

Date: June 24, 2026 | Branch: Emman | Audited by: Claude Code

Scope: Full backend (Laravel 13), frontend (React 19 + TypeScript), and all uncommitted changes

Severity	Count	Description
CRITICAL	**8**	Credential exposure, race conditions, IDOR, XSS, authorization bypass
HIGH	**14**	N+1 queries, missing pagination, file upload spoofing, rate limiting, error boundaries
MEDIUM	**16**	Missing FK constraints, idempotency, type safety, loading states, status constants
LOW	**15**	Dead code, audit logging gaps, accessibility, unused imports, API contract drift

Top 5 immediate action items

1. **.env committed to git** — credentials exposed (Gmail password, DB password, app keys). Rotate everything and scrub history.

2. **Race condition in TripTicket control number** — lockForUpdate() doesn't extend to create(), concurrent requests can duplicate DTT-YYYY-NNNN.

3. **IDOR in CollectionController** — show() has zero authorization; any authenticated user can view any collection's payment data.

4. **Cash budget status bug** — validation accepts 'approved' but code immediately overwrites to 'pending_super_admin', causing state mismatch.

5. **Mail queue removal** — all 7 mail classes had ShouldQueue stripped; email sending now blocks the HTTP request thread.

Developer 1 — Backend security + data integrity

Focus: highest-risk security items — credential leak, IDOR, webhook tampering, file upload spoofing, rate limiting. Requires security mindset.

CRITICAL

CRIT	Remove .env from git history, rotate all exposed credentials (Gmail app password, DB password, app keys)	`backend/.env`
CRIT	Fix race condition in TripTicket control number generation — lock scope doesn't extend to create()	`InvoiceFinalizationService:320-337`
CRIT	Add timestamp freshness check to PayMongo webhook signature validation (±5 min window)	`BillingController:501-545`
CRIT	Fix IDOR in CollectionController::show() — no authorization check, any user can view any collection	`CollectionController:99-102`
CRIT	Add role-scoped filtering to CashBudgetRequestController::index() — returns ALL records to any user	`CashBudgetRequestController:13-16`

HIGH

HIGH	Add rate limiting to public portal upload, password reset, and webhook endpoints	`routes/api.php`
HIGH	Add MIME-type validation (magic bytes) to base64 avatar upload — regex check is spoofable	`ProfileController:58-76`
HIGH	Validate custom_documents array in accreditation — bypasses server-side validation entirely	`AccreditationController:76-95`
HIGH	Add expiry check to legacy kyc_token accreditation flow — tokens valid indefinitely	`AccreditationController:258-280`
HIGH	Add race condition protection to trip ticket conflict-check (pessimistic lock or unique constraint)	`routes/api.php:141-190`

MEDIUM

MED	Add FK constraint for payroll_cycle_id in liquidations migration + DB-level check on source_type enum	`Migration 2026_06_24_100002`
MED	Add inverse belongsTo('liquidation') on CashBudgetRequest model — currently asymmetric	`CashBudgetRequest.php`
MED	Verify CORS config is not set to * — restrict to explicit frontend origins only	`config/cors.php`
MED	Remove PII from public drivers endpoint — currently exposes first/last names	`routes/api.php:74-103`

LOW

LOW	Delete dead test/debug scripts from backend root (check_john.php, test.php, reset.php, etc. ~10 files)	`backend/ root`

LOW	Add audit logging to CollectionController and CommissionController mutating endpoints	`CollectionController, CommissionController`

Developer 2 — Backend logic + workflow fixes

Focus: cash budget two-tier approval flow, N+1 queries, pagination gaps, test coverage. Requires domain knowledge of business workflows.

CRITICAL

CRIT	Fix cash budget status auto-forwarding bug — 'approved' is validated then immediately overwritten to 'pending_super_admin'	`CashBudgetRequestController:132-134`
CRIT	Restore ShouldQueue on all 7 mail classes — removal causes synchronous email sending, blocking requests	`All Mail classes (7 files)`

HIGH

HIGH	Fix N+1 query explosion in DashboardController::peakClientActivityData() — loads ALL job orders into memory	`DashboardController:38-98`
HIGH	Add pagination to /dashboards/approvals endpoint — currently loads unbounded records	`DashboardController:641-710`
HIGH	Add pagination to CollectionController::index() — returns full table with no limit	`CollectionController:38`
HIGH	Add pagination to JobOrderController::availableSupplies() — returns all inventory at once	`JobOrderController:297-337`

MEDIUM

MED	Add idempotency handling to LiquidationController::store() and PublicRequestActionController::approve()	`LiquidationController, PublicRequestActionController`
MED	Extract status constants to class constants — replace magic strings across all controllers	`CashBudgetRequest, Liquidation, TripTicket`
MED	Standardize pagination defaults across all endpoints (currently mixed: 20, 50, or none)	`All controllers`
MED	Add missing test cases: reject workflow, pending_super_admin → disbursed transition, liquidation CRUD	`AccountingTest.php`
MED	Add bus_id existence validation before creating TripTicket in invoice finalization	`InvoiceFinalizationService`

LOW

LOW	Standardize mail exception handling — some controllers suppress, some don't	`All controllers that send mail`
LOW	Add return type hints to all controller public methods (~60+ methods missing)	`All controllers`
LOW	Regenerate OpenAPI specs — auth.yaml role enum lists 5 roles, actual codebase has 13+	`api-contracts/*.yaml`
LOW	Add test coverage for ~15 untested controllers (Accounting/*, ContractController, etc.)	`backend/tests/`

Developer 3 — Frontend security + UX

Focus: XSS fixes, error boundaries, RoleGuard permission checks, loading states, accessibility. Can work fully in parallel with D1/D2.

CRITICAL

CRIT	Remove dangerouslySetInnerHTML from print CSS in Payroll.tsx and SalesCheckout.tsx — replace with static style tags	`Payroll.tsx:928-995, SalesCheckout.tsx:928-995`

HIGH

HIGH	Add global ErrorBoundary component wrapping all routes — unhandled errors currently crash the app	`App.tsx`
HIGH	Add comprehensive error handling to API client interceptor — handle network errors, 5xx, timeouts	`api/client.ts:46-48`
HIGH	Fix RoleGuard to check module-level permissions via hasPermission(), not just role membership	`guards/RoleGuard.tsx`
HIGH	Sanitize localStorage values for portal branding — validate logo/bg URLs against allowlist	`Login.tsx:62-90`

MEDIUM

MED	Add loading/disabled states to all form submission buttons to prevent double-submit	`Login.tsx, SalesCheckout.tsx, Payroll.tsx`
MED	Add null/undefined guards on all API response nested property access	`Multiple pages`
MED	Replace native confirm() dialogs with styled ConfirmDialog component for destructive actions	`Payroll.tsx:799, admin pages`
MED	Add request timeout (30s) to axios client config	`api/client.ts`
MED	Type custom_permissions properly in auth.ts — currently 'any', enables silent privilege escalation	`types/auth.ts:75`

LOW

LOW	Improve email validation regex to RFC 5322 compliance + support international phone formats	`SalesCheckout.tsx:166-175`
LOW	Add ARIA attributes (aria-label, aria-describedby, role) to all interactive elements	`All form inputs and buttons`
LOW	Fix Login slideshow memory leak — slideDuration in useEffect deps causes interval restarts	`Login.tsx:130-137`
LOW	Standardize error message display — some use err.response?.data?.message, others generic text	`All pages with API calls`
LOW	Run ESLint --fix to remove unused imports across all pages	`Payroll.tsx, SalesCheckout.tsx, etc.`

Developer assignment rationale

Developer	Focus area	Tasks	Rationale
D1	Backend security + data integrity	16	Highest-risk items: credential leak, IDOR, webhook tampering, file upload spoofing, rate limiting. Needs security mindset.
D2	Backend logic + workflow	15	Cash budget two-tier approval flow, N+1 queries, pagination gaps, test coverage. Needs domain knowledge of business workflows.
D3	Frontend security + UX	15	XSS fixes, error boundaries, RoleGuard permission checks, loading states, accessibility. Can work fully in parallel with D1/D2.

Key notes

• All three workstreams are independent — no blocking dependencies between developers.

• Each developer should start with their CRITICAL items and work down by severity.

• The .env credential rotation (D1 task #1) should be done within hours, not days.

• The mail queue restoration (D2 task #2) should be verified as intentional before reverting — if it was deliberate (e.g., debugging), document why.

• Frontend work (D3) can proceed fully in parallel since it touches no backend files.

• After all critical/high items are resolved, run the full test suite (php artisan test) to verify no regressions.

Positive patterns observed

• Strong use of Laravel validation (FormRequest classes) in most controllers

• Proper use of pessimistic locking in financial transactions

• Audit logging implemented in key endpoints via AuditLogger middleware

• 2FA TOTP setup and verification fully implemented

• Soft deletes on sensitive models prevent accidental data loss

• Database transactions used for multi-step operations

• New helper methods (determineBudgetSourceType, resolveDebitAccountAndDescription) reduce duplication well